

Research Summary: Secure Sorting and Selection via Function Secret Sharing

Paper by: Amit Agarwal, Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Mahimna Kelkar, and Yiping Ma (2024)

Hitarth Makawana

Roll No: 230479

hitarthkm23@iitk.ac.in

Ronav Puri

Roll No: 230815

ronavgp23@iitk.ac.in

March 12, 2026

Premise

The paper addresses efficiency bottlenecks in secure multi-party sorting. While traditional Batcher's networks require $O(\log^2 n)$ rounds, this work utilizes **Function Secret Sharing (FSS)** to collapse operations into constant rounds. The core logic shifts from a sequential network to a "Ranking and Routing" paradigm using succinct FSS keys.

- **Framework:** $(2 + 1)$ -party or 3-party honest-majority settings.

Core Technical Building Block: FSS

Function Secret Sharing (FSS) allows a function f from a known family to be shared into keys (k_0, k_1) . When a party P_i evaluates their key on an input x , they obtain a share y_i such that $y_0 + y_1 = f(x)$.

- **Distributed Comparison Function (DCF):** A specialized FSS where $f_{\alpha,\beta}(x)$ outputs β if $x < \alpha$, and 0 otherwise i.e. Let $f_{\alpha,\beta}^<(x) = \beta$ if $x < \alpha$ and 0 otherwise. This is the "engine" for secure comparisons.
- **Distributed Point Function (DPF):** A function that outputs non-zero only at a single point α . Let $f_{\alpha,\beta}^=(x) = \beta$ if $x = \alpha$ and 0 otherwise. This is used for oblivious indexing and routing.

Detailed Protocol Explanations

1. The CmpAgg (Compare-Aggregate) Protocol

This is the primary protocol for ranking elements in a large domain. Instead of a sequential network of gates, it uses a parallel "all-to-all" comparison strategy that is made efficient through FSS.

- **Mechanism:** For a set of n secret-shared inputs $[x_1], \dots, [x_n]$, the parties use FSS to generate $\binom{n}{2}$ DCF keys. For every pair (x_i, x_j) , the DCF evaluates the comparison $x_i < x_j$.
- **Local Aggregation:** Since the outputs of the DCFs are additive secret shares, each party can locally sum the results. To find the rank of x_i , a party sums all shares resulting from the comparisons of x_i against every other x_j . Mathematically, The rank r_i for element x_i is defined as:

$$r_i = \sum_{j=1, j \neq i}^n \mathbf{1}_{x_j < x_i} + \sum_{j < i} \mathbf{1}_{x_j = x_i} \quad (1)$$

Where $\mathbf{1}$ is the indicator function computed via DCF.

Algorithm 1 FSS-based Ranking (CmpAgg)

```
1: Input: Secret shares  $[x_1], \dots, [x_n]$ 
2: Setup: Dealer provides DCF keys  $k_{i,j}$  for comparisons  $x_i < x_j$ 
3: for each party  $P_\sigma \in \{P_0, P_1\}$  do
4:   for  $i, j \in \{1 \dots n\}$  do
5:      $[b_{i,j}]_\sigma \leftarrow \text{FSS.Eval}(k_{i,j}, [x_i]_\sigma, [x_j]_\sigma)$ 
6:   end for
7:    $[r_i]_\sigma \leftarrow \sum_{j=1}^n [b_{i,j}]_\sigma$  {Local summation of shares}
8: end for
9: Output: Secret shares of ranks  $[r_1], \dots, [r_n]$ 
```

- **Analysis:** This protocol requires only 1 online round to exchange the comparison results. Even though there are $O(n^2)$ logical comparisons, the communication scales only with the number of inputs n and the succinct FSS key size, rather than the total number of comparisons.

2. The mult-DPF Protocol (Small Domains)

When input domain is limited (e.g., 10-bit or 16-bit integers), the authors provide an even more efficient approach using DPFs that avoids the $O(n^2)$ overhead entirely. Typically, for domain size $M = 2^L$, we use the linearity of DPFs to build a shared histogram.

- **Mechanism:** Each party uses their shares of the inputs to evaluate a DPF. This creates a "shared histogram" of the entire dataset across the small domain. For each input x_i , parties evaluate a DPF $f_{x_i,1}^\pm(x)$. The collective histogram H is:

$$[H(v)] = \sum_{i=1}^n \text{DPF.Eval}(k_i, v) \quad \forall v \in \{0, \dots, M-1\} \quad (2)$$

where k_i is the succinct key of DPF $f_{x_i,1}^\pm(x)$.

- **Prefix Summation:** By performing a local prefix-sum on this shared histogram, the parties obtain the secret-shared rank for every possible value in the domain. The rank $[r_i]$ is then the prefix sum: $[r_i] = \sum_{v=0}^{x_i-1} [H(v)]$.
- **Result:** This protocol is exceptionally fast for "narrow" data (like ages, ratings, or small categories) because the communication depends only on the domain size and n , making it practically linear in many settings.

3. Shuffle based sorting and selection

The original elements are secretly shuffled using a random permutation π as a part of pre-processing and then based on the shuffled list, rank vector $[v]$ is calculated using the ranking protocol described above. This rank vector $[v]$ is then made public. Trivial routing is done by each party using this rank vector $[v]$.

4. Three-Round Routing Protocol

Once the secret-shared ranks are computed, the protocol must move the payloads to their sorted positions without revealing the ranks to any party. The following 3-phase process utilizes the properties of Distributed Point Functions (DPF) to perform this oblivious permutation.

Phase 1: Pre-Processing

Let the shuffled list consist of elements where the i -th element has a secret-shared payload $[z_i]$ and a secret-shared rank $[y_i]$.

- **Random Masks:** For each index i , a random mask r_i is assigned and secret-shared among the parties.
- **DPF key:** For each index i , a DPF key is generated for the function $f_{r_i,1}^{\equiv}(x)$. The keys for these n DPFs are distributed to the parties via FSS.

Phase 2: Online Evaluation (Round 2)

Parties interact to compute the public shifted ranks, which allows for oblivious indexing.

1. **Masked Rank Reconstruction:** Each party computes their share $[y_i] + [r_i] = [\hat{y}_i]$. These shares are broadcasted and reconstructed so that $\hat{y}_i = y_i + r_i$ becomes public for all $i \in [n]$.
2. **Pointer Vector Construction:** For each destination index $k \in [n]$, parties evaluate their DPF shares at the shifted values. The share of the i -th component of the pointer vector for the k -th ranked element, denoted $[e_{k,i}]$, is computed as:

$$[e_{k,i}] = f_i(\hat{y}_i - k) \quad (3)$$

Summing these across parties results in a "one-hot" pointer vector e_k where the entry is 1 only if the i -th element's rank y_i is equal to k (i.e., when $\hat{y}_i - k = r_i$).

Phase 3: Payload Selection (Round 3)

The final step maps the payloads to the sorted output using the pointer vectors.

- **Selection (Top- k):** To retrieve the payload of the element with rank k , parties compute the secret-shared dot product of the payload vector $\vec{z}$ and the pointer vector $\vec{e}_k$:

$$[\text{Sorted}_k] = \sum_{i=1}^n [z_i] \cdot [e_{k,i}] \quad (4)$$

- **Sorting:** For a full sort, the selection process is performed in parallel for every $k \in \{1, \dots, n\}$.

Algorithm 2 DPF-based Oblivious Routing

- 1: **Inputs:** Shared payloads $[z_i]$, Shared ranks $[y_i]$, Shared DPF keys $f_i(r_i) = 1$.
 - 2: **Round 2 (Public):** Reconstruct $\hat{y}_i \leftarrow [y_i] + [r_i]$ for all i .
 - 3: **for** each rank $k \in [n]$ **do**
 - 4: **for** each element $i \in [n]$ **do**
 - 5: $[e_{k,i}] \leftarrow \text{DPF.Eval}(f_i, \hat{y}_i - k)$
 - 6: **end for**
 - 7: $[\text{Output}_k] \leftarrow \sum_{i=1}^n [z_i] \cdot [e_{k,i}]$ {Compute k -th sorted element}
 - 8: **end for**
 - 9: **Output:** Sorted shared sequence $[\text{Output}_1, \dots, \text{Output}_n]$.
-

5. Two-Round Routing (DPF-based)

This protocol is just an improvement over the three-round routing protocol, which removes the necessity of calculating the ranks beforehand for DPF key calculations. The 2nd round maps elements to their sorted positions using a "blind" write mechanism.

- **Round 1:** The parties generate a random mask s_i and mask the elements with s_i .

$$x'_i = s_i \cdot x_i$$

Now the parties use a modified version of the *DPF.Eval* function wherein the function, now instead of producing 1 as the output, it produces s_i^{-1} . Which is later used in the "blind" write mechanism.

- **Round 2:** The authors use a DPF where the "point" α is the secret-shared rank. To place x_i at its rank r_i , we evaluate a DPF keyed at the secret value r_i to "pick" the correct element for each sorted position without ever learning what the ranks actually were. The j -th element of the sorted array Y is:

$$[y_j] = \sum_{i=1}^n \text{DPF.Eval}(k_{r_i}, j) \cdot [x'_i] \quad (5)$$

Since *DPF.Eval* results in shares of 1 if $r_i = j$ and 0 otherwise, the summation correctly selects the appropriate x_i . This "blind" routing allows the entire sort to complete in just 2 rounds (one for ranking, one for routing).

Implementation & Results

- **Implementation Settings:** The authors evaluated their protocols using a C++ implementation built on the `Emp-toolkit` and `Porthos` frameworks.
- **Performance:** Reaches the theoretical lower bound of **2 rounds** for sorting.
- **Key Result:** Communication costs are $O(n \cdot \lambda)$ where λ is the security parameter, providing up to 14× speedup over previous circuit-based approaches.

For sorting $n = 2^{10} = 1024$ secret shared 32-bit integers in a 2 + 1 party semi-honest setting, this work shows the following results:

- **Rounds:** 2 – 3
- **Communication bandwidth:** 1.8MB
- **Time taken in LAN setting:** 0.04s
- **Time taken in WAN setting:** 0.31s